Week 4, Day 1: Kafka Producers in Depth and Heap Problems

Prepared for Upendra Kumar Sharma | 3.5 to 4 hours | Main topic: Kafka producer internals, acks, idempotence, retries, partitioning, ordering, transactions | DSA: Heaps and priority queues

Today's Schedule

- **0:00 to 0:40** Study how a producer works internally and how batching affects throughput (sections 1 and 2).
- **0:40 to 1:20** Study reliability: acks, retries, timeouts, idempotence and topic-level durability (sections 3 and 4).
- **1:20 to 1:50** Study partitioning, ordering and serialization (sections 5 and 6).
- **1:50 to 2:15** Study producer transactions and Spring Kafka usage (sections 7 and 8).
- **2:15 to 3:15** DSA: solve the 3 heap problems in section 10. Top K Frequent is a revisit from Week 1, so spend your time on the other two.
- **3:15 to 3:45** Answer the interview questions aloud (section 11) and add 10 flashcards.
- **Optional (60 to 90 min):** do the hands-on experiments in section 9 and connect your outbox relay to Kafka.

This guide builds on Week 3, Day 5. Review that section first if acks, partitions or keys feel unfamiliar.

1. What Happens When You Call send()

1. **Serialize** the key and value into bytes.
2. **Choose a partition:** from the record's explicit partition, or a partitioner (a hash of the key, or a sticky choice when there is no key).
3. **Append to a batch** in the producer's in-memory **record accumulator**, one batch per partition. This call returns **immediately** with a Future. Nothing has been sent yet.
4. A background **sender thread** collects ready batches and sends them to the **leader of each partition**. A batch is ready when it is full (**batch.size**) or when it has waited long enough (**linger.ms**).
5. The broker writes the batch, replicates it according to the topic settings, and sends back an acknowledgement according to **acks**.
6. The Future completes and your **callback** runs, either with record metadata (topic, partition, offset) or an exception.

What this means in practice

- send() is **asynchronous**. If you ignore the result, a failure goes unnoticed. Always check the callback or the Future, at least by logging, and for important data by waiting.
- The accumulator has a limited size (**buffer.memory**, 32 MB by default). If the application produces faster than the broker accepts, the buffer fills and send() **blocks for up to max.block.ms** and then throws a TimeoutException. That is back-pressure reaching your code.
- **Always close or flush** the producer on shutdown (flush() sends pending batches), otherwise buffered records are lost.
- A producer instance is **thread-safe and should be shared**. Creating one per request wastes connections and defeats batching.

2. Batching, Compression and the Throughput and Latency Trade-Off

- **linger.ms:** how long the producer waits to fill a batch before sending. A value of 0 sends as soon as possible (many small batches). A few to tens of milliseconds gives bigger batches. It is the main throughput knob. Defaults differ between Kafka versions (0 in 3.x, 5 ms from 4.0), so check yours.
- **batch.size:** maximum bytes per batch for a partition (16 KB by default). Larger batches mean fewer requests. They only fill if traffic is high enough or linger.ms is long enough.
- **compression.type:** none, gzip, snappy, lz4 or zstd. Compression works on whole batches, so **bigger batches compress better**. lz4 and zstd are common choices. It saves network and disk, and costs a little CPU.
- **Rule of thumb:** raise linger.ms (for example 5 to 20 ms), raise batch.size (for example 64 KB), and turn on lz4 or zstd for **throughput**. Keep linger.ms low for **latency-sensitive** paths.
- **Message size:** the default limit is about 1 MB (max.request.size on the producer, message.max.bytes on the broker). Do not send large files through Kafka. Store them in object storage and send a **reference** (the claim check pattern).

3. Reliability: acks, Retries and Timeouts

acks

- **0:** fire and forget. Fastest, and silent data loss is possible.
- **1:** the partition leader has written it. If the leader fails before followers copy the record, the record is lost.

- **all (or -1):** every replica in the **in-sync replica set (ISR)** has it. This is the safe choice, and the default in recent versions.

The trap to know: acks=all means all **current ISR members**, not all replicas. If the ISR has shrunk to the leader alone, acks=all behaves like acks=1. To prevent that, set **min.insync.replicas** on the topic (or broker). Then, when the ISR is smaller than that number, the write is **rejected** with NotEnoughReplicasException instead of being accepted unsafely.

A durable setup: replication factor 3, min.insync.replicas 2, acks=all, idempotence on, and **unclean.leader.election.enable=false** (a replica that is out of sync must never become leader, because that would lose committed records). With this setup the cluster tolerates the loss of one broker without data loss and keeps accepting writes. If two brokers fail, writes stop. That is the intended trade-off of availability for durability.

Retries and timeouts

- **retries** defaults to a very large number in recent versions, and the real limit is **delivery.timeout.ms** (2 minutes by default): the total time from send() to success or final failure, including batching, retries and waiting.
- **request.timeout.ms** (30 seconds by default) is how long the producer waits for **one request** to be answered before retrying it.
- delivery.timeout.ms must be at least linger.ms plus request.timeout.ms.
- **Retriable errors** (a leader election in progress, a network blip, NotEnoughReplicas) are retried automatically. **Non-retriable errors** (RecordTooLargeException, a serialization error, authorization failure) fail immediately, and you must handle them.
- After the delivery timeout the callback gets an exception. Decide what happens next: log and alert, retry from a durable source (for example your **outbox**), or route to a failure store. Silent dropping is the bug to avoid.

4. Idempotent Producer

The problem: a retry after a lost acknowledgement can write the **same record twice**, and with several requests in flight a retry can also **reorder** records.

How idempotence fixes it

- On start, the broker gives the producer a **producer id (PID)** with an epoch.
- The producer attaches an increasing **sequence number** to each batch, **per partition**.
- The broker remembers the last sequences for each PID and partition. It **drops duplicates** and rejects out-of-order batches, so retries become safe and order is kept.
- It needs acks=all, a retries value above 0, and max.in.flight.requests.per.connection of **5 or less**. It is enabled by default in current versions.

Limits to state honestly

- Idempotence covers **one producer session** and one partition at a time. If the application restarts it gets a new PID, so a resend of the same business record after a restart is **not** deduplicated. That is why you still need **business-level idempotency** (an event id and idempotent consumers, Week 3, Day 4).
- It does not make your **database or HTTP** side effects idempotent.

5. Partitioning and Ordering

How the partition is chosen

- If you specify a partition, that one is used.
- If there is a **key**, the default partitioner hashes it (murmur2) and takes it modulo the partition count. **The same key always goes to the same partition.**
- With **no key**, the producer uses a **sticky** approach: it fills a batch for one partition, then switches to another. That gives good batching and an even spread over time. It is not strict round robin.

What order is guaranteed

- Per partition, in the order records were **appended**. There is no ordering across partitions.
- For one key to stay ordered, all of the following must hold: the key maps to one partition, the partition count has **not changed**, the producer is idempotent (or has max.in.flight set to 1), and **one producer** writes that key (two producers writing the same key interleave unpredictably).
- On the consumer side, a partition is read by one consumer thread at a time, so order is preserved as long as you process each record in order. Handing records to a thread pool breaks it.

Choosing a key

- Use the **entity whose events must be ordered**: orderId, accountId, customerId.
- **Hot keys and skew:** if one key carries a large share of traffic (a very large customer, or a null placeholder key), its partition becomes a bottleneck while others sit idle. Check the partition distribution. Fixes: pick a finer-grained key, add a **salt** (for example key plus a small bucket number) when ordering can be relaxed to that bucket, or write a **custom Partitioner** for special cases.
- **Increasing the partition count** changes the mapping for new records, so a key can move to a different partition, and for a time its older and newer records are in different partitions. Plan the partition count generously at the start (consider peak consumer parallelism), and avoid changing it for topics where ordering matters.

6. Serialization and Schemas

- **String and JSON:** easy to start with and to debug. No enforced contract, larger messages, and field type mistakes appear at runtime.
- **Avro, Protobuf or JSON Schema with a Schema Registry:** compact messages, an enforced contract, and **compatibility checks** when schemas change. The registry stores each schema, and records carry a small schema id.
- **Compatibility modes:** BACKWARD (new schema can read old data, so you can upgrade consumers first), FORWARD (old consumers can read new data), and FULL (both). Typical safe changes: add a field with a default, never reuse or retype a field.
- **Headers:** put metadata such as eventId, eventType, a correlation id and the schema version in record headers instead of the payload where it makes sense.
- **Event design tips:** include eventId, occurredAt, a type and a version. Prefer events that describe facts (OrderCreated) with enough data for consumers, rather than forcing a call back to the producer.

7. Producer Transactions (Exactly-Once Inside Kafka)

What they give you: write to **several partitions and topics atomically**, and combine consuming and producing into one atomic step (consume a record, produce results, commit the consumer offset, all or nothing).

How it works

- Set a **transactional.id**. Call initTransactions once, then beginTransaction, send the records, optionally sendOffsetsToTransaction, then commitTransaction or abortTransaction.
- A **transaction coordinator** (a broker) tracks the state in an internal topic. Records in an aborted transaction stay in the log but are **hidden** from consumers that use **isolation.level=read_committed**.
- **Zombie fencing:** a new producer with the same transactional.id bumps the epoch, and the old instance is fenced off, so a stalled instance cannot write after its replacement has started.

Limits and costs

- It covers **Kafka reads and writes only**. A database insert or an email in the middle is outside the transaction.
- It adds latency and complexity, and consumers must use read_committed (which waits for the transaction to finish).

- **When to use it:** stream processing pipelines that read from a topic and write to another (Kafka Streams uses it for exactly-once). For typical microservice flows, **the outbox pattern plus idempotent consumers** is simpler and more common.

8. Producing from Spring Boot

Configuration (application.yml)

```
spring:
  kafka:
    bootstrap-servers: kafka:19092
    producer:
      acks: all
      compression-type: lz4
      batch-size: 65536
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer:
org.apache.kafka.common.serialization.StringSerializer
      properties:
        enable.idempotence: true
        linger.ms: 10
        delivery.timeout.ms: 30000
        max.in.flight.requests.per.connection: 5
```

Sending from the outbox relay, with a key, a header and a bounded wait

```
@Component
public class OutboxPublisher {
    private final KafkaTemplate<String, String> kafka;
    public OutboxPublisher(KafkaTemplate<String, String> kafka) { this.kafka
= kafka; }

    public void publish(OutboxEvent event) throws Exception {
        ProducerRecord<String, String> record =
                new ProducerRecord<>("orders.created",
event.getAggregateId(), event.getPayload());
        record.headers().add("eventId",
                event.getId().toString().getBytes(StandardCharsets.UTF_8));

        // wait for the broker acknowledgement, so we only mark the row
published on success
        kafka.send(record).get(10, TimeUnit.SECONDS);
    }
}
```

- The relay marks the outbox row published **only after** the acknowledgement. A failure leaves the row for the next attempt, which gives at-least-once delivery.
- Waiting on each send is simplest. For higher throughput, send a batch of records, collect the futures, then wait for all of them and mark the successful ones.

Transactions in Spring (when you need them)

```
spring:
  kafka:
    producer:
      transaction-id-prefix: order-tx-
```

```
kafka.executeInTransaction(ops -> {
    ops.send("orders.confirmed", orderId, payload);
    ops.send("notifications.requested", orderId, notification);
    return true;                        // both records become visible
together, or neither does
});
```

Monitoring the producer (Micrometer and JMX): record-send-rate, record-error-rate, request-latency-avg, batch-size-avg, compression-rate-avg and buffer-available-bytes. A falling buffer-available-bytes means the producer is being throttled. A rising record-error-rate needs an alert.

9. Optional Hands-On (60 to 90 Minutes)

Use the Compose setup from Week 3, Day 6. Script names can vary by version, so check the tools in /opt/kafka/bin.

1. **Throughput experiment.** Create a topic perf with 3 partitions, then run the performance tool with different settings and record the records per second and the average latency each time.

```
docker compose exec kafka /opt/kafka/bin/kafka-producer-perf-test.sh --topic
perf --num-records 1000000 --record-size 500 --throughput -1 --producer-props
bootstrap.servers=localhost:9092 linger.ms=0 batch.size=16384
docker compose exec kafka /opt/kafka/bin/kafka-producer-perf-test.sh --topic
perf --num-records 1000000 --record-size 500 --throughput -1 --producer-props
bootstrap.servers=localhost:9092 linger.ms=20 batch.size=65536
compression.type=lz4
```

2. **Skew experiment.** Write a small producer that sends 10,000 records where 90 percent share one key. Check the end offset of each partition (kafka-get-offsets.sh in recent versions) and see the imbalance. Then change the key and compare.

3. **Failure experiment.** Send with a callback, stop the Kafka container, and watch how long it takes for the callback to report a timeout. Match it to delivery.timeout.ms.
4. **Acknowledgement and metrics.** Print the partition and offset from the callback for 20 records. Note that records with the same key land on the same partition with increasing offsets.
5. **Transaction experiment.** Send two records inside a transaction and abort it. Confirm that a consumer with isolation.level=read_committed sees nothing, and one with read_uncommitted sees the records.
6. **Project step.** Connect your **outbox relay** from Week 3, Day 6 to Kafka using the publisher above, with the order id as the key. Create an order through the gateway and watch the event arrive on orders.created (use the console consumer). Record what happens if Kafka is down when the order is placed, and how the relay catches up when Kafka returns.

Note your numbers. A line such as "batching with lz4 raised throughput from X to Y records per second" is a strong interview detail.

10. DSA: Heaps and Priority Queues (3 Problems)

Java facts

- PriorityQueue is a **binary min-heap** by default. For a max-heap, pass Comparator.reverseOrder() or a custom comparator.
- offer and poll are **O(log n)**, peek is **O(1)**, and remove(Object) is **O(n)**. Building a heap from a collection with the constructor is **O(n)** (heapify).
- Iterating a PriorityQueue does **not** give sorted order.
- **Pattern:** to keep the k largest items, use a **min-heap of size k** (evict the smallest). To keep the k smallest, use a max-heap of size k. The heap top is always the boundary element.

Problem 1: Kth Largest Element in an Array

- **Idea:** keep a min-heap of size k. Add each number, and if the heap grows beyond k, remove the smallest. At the end the top is the kth largest.
- **Complexity:** O(n log k) time, O(k) space. **Quickselect** averages O(n) time with O(1) extra space but has an O(n squared) worst case, so mention it as the alternative.
- **Edge cases:** k equal to 1 (the maximum), k equal to the array length (the minimum), duplicate values (they count separately).

```
public int findKthLargest(int[] nums, int k) {
    PriorityQueue<Integer> heap = new PriorityQueue<>();
    for (int x : nums) {
        heap.offer(x);
        if (heap.size() > k) heap.poll();
    }
    return heap.peek();
}
```

Problem 2: Top K Frequent Elements (revisit with a heap)

- **Idea:** count frequencies in a HashMap, then keep a min-heap of size k ordered by frequency. In Week 1 you may have used bucket sort, which is O(n). Be able to give both and compare them.
- **Complexity:** O(n log k) time with the heap, O(n) space.
- **Edge cases:** k equal to the number of distinct values, all values equal, negative numbers.

```
public int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    for (int x : nums) freq.merge(x, 1, Integer::sum);

    PriorityQueue<Map.Entry<Integer, Integer>> heap =
            new PriorityQueue<>(Map.Entry.comparingByValue());
    for (Map.Entry<Integer, Integer> e : freq.entrySet()) {
        heap.offer(e);
        if (heap.size() > k) heap.poll();
    }

    int[] result = new int[k];
    for (int i = k - 1; i >= 0; i--) result[i] = heap.poll().getKey();
    return result;
}
```

Problem 3: Merge K Sorted Lists

- **Idea:** put the head of each list into a **min-heap ordered by value**. Repeatedly remove the smallest node, append it to the result, and push that node's next node. The heap never holds more than k nodes.
- **Complexity:** O(N log k) time for N total nodes, O(k) space. **Alternative:** merge the lists in pairs (divide and conquer) using the merge-two-lists code from Week 2, also O(N log k) time, with O(1) extra space.

- **Edge cases:** an empty array of lists, null entries inside the array, lists of very different lengths.
- **Link to Kafka:** this is how a consumer merges several ordered partitions into one time-ordered stream.

```
public ListNode mergeKLists(ListNode[] lists) {
    if (lists == null || lists.length == 0) return null;
    PriorityQueue<ListNode> heap = new PriorityQueue<>((a, b) ->
Integer.compare(a.val, b.val));
    for (ListNode node : lists) {
        if (node != null) heap.offer(node);
    }
    ListNode dummy = new ListNode(0), tail = dummy;
    while (!heap.isEmpty()) {
        ListNode node = heap.poll();
        tail.next = node;
        tail = node;
        if (node.next != null) heap.offer(node.next);
    }
    return dummy.next;
}
```

11. Interview Questions to Answer Aloud

1. Describe what happens from the moment you call send() until the record is acknowledged.
2. Why is send() asynchronous, and what happens if you ignore the returned Future?
3. What do linger.ms, batch.size and compression.type do? How would you tune for throughput and for latency?
4. What do acks 0, 1 and all mean? Why can acks=all still lose data without min.insync.replicas?
5. What configuration gives durability when one broker fails, and why does unclean leader election matter?
6. How do retries, request.timeout.ms and delivery.timeout.ms relate?
7. What does the idempotent producer do, and what are its limits after a restart?
8. How does Kafka guarantee ordering, and what can break it (retries, several producers, a change in the partition count)?
9. How would you detect and fix a hot partition?
10. How would you choose a partition key for orders, payments and notifications?

11. When would you use a Schema Registry, and what are BACKWARD and FORWARD compatibility?
12. What are Kafka transactions, what do they cover, and what do they not cover? What is zombie fencing?
13. How do you publish reliably from a service that also writes to a database?
14. What happens when the producer buffer fills up?

12. End-of-Day Checklist

- [ ] I can narrate the path of a record from send() to acknowledgement, including the buffer and sender thread.
- [ ] I can tune batching and compression and explain the trade-offs.
- [ ] I can state a durable configuration and explain min.insync.replicas and the acks=all trap.
- [ ] I can explain idempotence (PID, sequence numbers) and its limits.
- [ ] I can explain the ordering conditions, hot keys and the effect of changing partitions.
- [ ] I can explain producer transactions and when to prefer the outbox pattern.
- [ ] I solved all 3 heap problems and can state the k-largest pattern.
- [ ] (Optional) My outbox relay publishes to Kafka, and I recorded what happens when Kafka is down.
- [ ] I wrote at least 10 flashcards from questions I got wrong.

Tomorrow (Week 4, Day 2): Kafka consumers in depth (offsets, rebalancing, retry topics, dead letter queues, exactly-once consumption), plus Find Median from Data Stream and Task Scheduler.